Dovivo · Base de dades, API i spikes

Next.js 15 · tRPC 11 · Prisma 6 + PostgreSQL 17 · Auth.js v5

Esquema complet per al MVP a partir del Product Backlog (US-01 a US-25 i T-05) i del codi actual del repositori (`prisma/schema.prisma`, `src/server/api/routers`, `src/lib/validators`). Inclou els models de dades, els procediments de l'API i els resultats dels spikes T-07 (motor de cerca) i T-09 (xatbot).

Font de veritat: el model de dades és `prisma/schema.prisma` i cada canvi passa per una migració (`pnpm db:generate --name <canvi>`) que es puja al repositori juntament amb l'esquema. Mai `db:push` sobre bases de dades compartides. Aquest document descriu l'esquema objectiu; quan no coincideix amb el repo, la columna **Repo** ho indica.

Arquitectura en resum

Tema	Decisió
Aplicació	Una sola aplicació Next.js (App Router): frontend i backend al mateix projecte
API	tRPC a `/api/trpc`, amb tipus compartits de punta a punta (veure secció 7)
Base de dades	PostgreSQL 17 amb Prisma i migracions (`pnpm db:up` en local)
Identificadors	`String @id @default(cuid())`
Imports	Cèntims: `priceCents Int` (regla 4 d'AGENTS.md)
Enums	Enums natius de Prisma/PostgreSQL en MAJÚSCULES, replicats a zod
Noms	camelCase als models de Prisma
Autenticació	Auth.js v5: cookie de sessió JWT, proveïdor Credentials + OAuth opcional
Validació	Esquemes zod a `src/lib/validators`, compartits per API i formularis

1. Resum de models

Model (Prisma)	Per a què	Històries	Sprint	Repo
`User`	Comptes: particular, agència, admin	US-01..05, 07	S1	sí (amplia)
`Account`, `Session`, `VerificationToken`	Taules estàndard d'Auth.js (OAuth, verificació d'email)	US-02, 03, 06	S1	sí
`Property`	Anuncis de venda i lloguer	US-09, 10, 12, 17, 19, 20	S1-S2	sí (amplia)
`PropertyImage`	Fotos de cada anunci (màx. 15)	US-18	S2	sí (amplia)
`Conversation`	Fil entre interessat i anunciant per anunci	US-21, 22	S2	nou
`Message`	Missatges de cada conversa	US-21, 22, 23	S2	nou
`Favourite`	Anuncis desats per usuari	US-14	S3	sí
`Report`	Denúncies d'anuncis i la seva moderació	Rol admin, T-05	S3	nou
`UserPreference`	Preferències de cerca per defecte	US-15	S3*	nou
`ContactMessage`	Formulari de contacte amb Dovivo	US-25	S3*	nou
`PasswordResetToken`	Recuperar la contrasenya	US-08	Post	nou
`SavedSearch`	Cerques desades i alertes	US-16	Post	nou

Es recomana crear tots els models des del principi encara que la seva història sigui d'un sprint posterior: són petits i així l'esquema no canvia a mig projecte. El xatbot (US-24, Post) de moment no necessita cap model.

2. Relacions

Origen	Cardinalitat	Destí	Clau forana
`User`	1 : N	`Property`	`Property.ownerId`
`User`	1 : N	`Account`, `Session`	`userId`
`Property`	1 : N	`PropertyImage`	`PropertyImage.propertyId`
`User` ↔ `Property`	N : M	via `Favourite`	PK composta (`userId`, `propertyId`)
`Property`	1 : N	`Conversation`	`Conversation.propertyId`
`User`	1 : N	`Conversation`	`interestedUserId` i `advertiserId` (dues relacions amb nom)
`Conversation`	1 : N	`Message`	`Message.conversationId`
`User`	1 : N	`Message`	`Message.senderId`
`Property`	1 : N	`Report`	`Report.propertyId`
`User`	1 : N	`Report`	`reporterId` i `resolvedById` (dues relacions amb nom)
`User`	1 : 1	`UserPreference`	`UserPreference.userId` (és també la PK)
`User`	1 : N	`ContactMessage`, `SavedSearch`, `PasswordResetToken`	`userId`

Quan un model té dues relacions amb `User` (Conversation, Report), Prisma obliga a donar-los nom amb `@relation("...")`, p. ex. `@relation("ConversationInterested")` i `@relation("ConversationAdvertiser")`.

3. Convencions i enums

Convencions

- **Identificadors:** `String @id @default(cuid())`. A l'API viatgen com a string; es validen amb `z.string()`.
- **Imports:** sempre en cèntims d'euro (`Int`): `priceCents`, `minPriceCents`... El frontend els formata. Mai `Float` ni `Decimal` per a diners.
- **Dates:** `DateTime` (timestamp en UTC). `createdAt @default(now())` i `updatedAt @updatedAt`. superjson les transporta com a `Date` fins al client.
- **Coordenades:** `Float` (double precision), suficient per a cerca per radi i mapa.
- **Enums:** enums natius de Prisma en MAJÚSCULES, amb el mateix enum a zod a `src/lib/validators`. Afegir un valor requereix una migració (`ALTER TYPE ... ADD VALUE`, la genera Prisma) i actualitzar el `z.enum`.
- **Restriccions que Prisma no expressa** (CHECK, índexs únics parcials, extensions): s'afegeixen a mà al SQL de la migració, creant-la amb `pnpm db:generate --name <canvi> --create-only`, editant-la i aplicant-la.

Enums

Enum	Valors	Repo	Notes
`UserRole`	`USER` · `AGENT` · `ADMIN`	sí	USER = particular, AGENT = agència. Default USER. Ser anunciant no és un rol: qualsevol usuari pot publicar
`ListingType`	`SALE` · `RENT`	sí	Operació: venda o lloguer
`PropertyType`	`APARTMENT` · `HOUSE` · `STUDIO` · `ROOM` · `OFFICE` · `LAND` · `OTHER`	sí	**Proposta:** afegir `COMMERCIAL` (local) i `GARAGE` (plaça d'aparcament), que el backlog sí contemplava
`ListingStatus`	`DRAFT` · `PUBLISHED` · `RESERVED` · `CLOSED`	sí	DRAFT = creat o desactivat, no visible. CLOSED = venut o llogat: es distingeix amb `listingType` (CLOSED + SALE = venut, CLOSED + RENT = llogat). **Proposta:** afegir `BLOCKED`, que posa l'admin en moderar una denúncia
`ReportReason`	`FRAUD` · `WRONG_INFO` · `DUPLICATE` · `NOT_AVAILABLE` · `OFFENSIVE` · `OTHER`	nou	Motiu de la denúncia
`ReportStatus`	`PENDING` · `RESOLVED` · `DISMISSED`	nou	Default PENDING
`ContactStatus`	`OPEN` · `ANSWERED` · `CLOSED`	nou	Default OPEN

4. Detall de cada model

La columna **Repo** indica si el camp ja existeix al schema actual (**sí**) o s'ha d'afegir amb una migració (**nou**, fila ressaltada). Els límits de longitud i format es validen amb zod, no a la base de dades.

`User` US-01..07, admin

Camp	Tipus (Prisma)	Null	Repo	Notes
`id`	`String`	no	sí	PK, cuid
`name`	`String?`	sí	sí	Obligatori al registre (zod). Nullable perquè Auth.js/OAuth ho requereix
`email`	`String? @unique`	sí	sí	Únic, no es pot repetir (US-01). Inici de sessió (US-02). Obligatori per a Credentials via zod
`emailVerified`	`DateTime?`	sí	sí	US-06 (Post)
`image`	`String?`	sí	sí	Avatar (OAuth)
`role`	`UserRole`	no	sí	Default USER
`phone`	`String?`	sí	nou	Editable al perfil (US-05). Màx. 20 caràcters (zod)
`passwordHash`	`String?`	sí	nou	Hash de la contrasenya, mai en text pla (US-01). Null si l'usuari només entra amb OAuth. Mínim 8 caràcters (zod). Recomanat `bcryptjs`: és JS pur i no dona problemes de compilació a Windows
`blockedAt`	`DateTime?`	sí	nou	Si té valor, l'admin l'ha bloquejat: no pot iniciar sessió ni publicar
`createdAt` / `updatedAt`	`DateTime`	no	nou	`@default(now())` / `@updatedAt`

`Account`, `Session`, `VerificationToken` US-02, 03, 06

Models estàndard de l'adaptador de Prisma d'Auth.js, ja presents al repo; no s'han de modificar. `Account` guarda els comptes OAuth (Discord, Google...) vinculats a un usuari. `VerificationToken` es pot fer servir per a la verificació d'email (US-06).

Important: el proveïdor **Credentials** (email i contrasenya, US-01/02) d'Auth.js només funciona amb `session: { strategy: "jwt" }`. Amb JWT la taula `Session` no s'utilitza i tancar la sessió (US-03) és esborrar la cookie. Conseqüència: un usuari bloquejat manté el JWT fins que caduca, així que `protectedProcedure` ha de consultar `blockedAt` a la BD a cada petició (o fer servir una caducitat curta).

`Property` US-09, 10, 12, 17, 19, 20

Camp	Tipus (Prisma)	Null	Repo	Notes
`id`	`String`	no	sí	PK, cuid
`ownerId`	`String`	no	sí	FK → User.id (anunciant)
`title`	`String`	no	sí	5-120 caràcters (zod)
`description`	`String`	no	sí	zod actual: 20-5000. El backlog deia màx. 10.000: **cal decidir-ho**
`listingType`	`ListingType`	no	sí	SALE | RENT
`propertyType`	`PropertyType`	no	sí	Veure enums
`status`	`ListingStatus`	no	sí	Default DRAFT. `property.publish` el passa a PUBLISHED. CLOSED = US-20. BLOCKED (proposta) = moderació
`priceCents`	`Int`	no	sí	Cèntims d'euro, > 0 (zod). Int arriba fins a ~21 M€, suficient
`currency`	`String`	no	sí	Default 'EUR'
`areaM2`	`Int?`	sí	sí	> 0
`bedrooms / bathrooms`	`Int?`	sí	sí	≥ 0
`floor`	`String?`	sí	nou	Text lliure ('baixos', '3', 'àtic')
`address`	`String`	no	sí	
`neighborhood`	`String?`	sí	nou	Barri, per cercar per barri (US-10)
`city`	`String`	no	sí	
`province`	`String`	no	nou	
`postalCode`	`String?`	sí	sí	
`country`	`String`	no	sí	Default 'ES'
`latitude / longitude`	`Float?`	sí	sí	Cerca per radi i mapa (US-13)
`hasElevator / hasParking / isFurnished`	`Boolean`	no	nou	Default false
`createdAt / updatedAt`	`DateTime`	no	sí	
`publishedAt`	`DateTime?`	sí	sí	Data de publicació; ordre per defecte del llistat
`deletedAt`	`DateTime?`	sí	nou	Esborrat lògic (US-19). Desapareix de la cerca, però qui el tenia a preferits veu que ja no està disponible (US-14) i els xats es conserven

`PropertyImage` US-18

Els fitxers van a l'emmagatzematge d'imatges (T-10); aquí només es desa la ruta.

Camp	Tipus (Prisma)	Null	Repo	Notes
`id`	`String`	no	sí	PK, cuid
`propertyId`	`String`	no	sí	FK → Property.id
`url`	`String`	no	sí	Clau o URL de l'original a l'emmagatzematge
`thumbnailUrl`	`String?`	sí	nou	Versió optimitzada per al llistat i la fitxa
`alt`	`String?`	sí	sí	Text alternatiu (accessibilitat)
`mimeType`	`String`	no	nou	image/jpeg | image/png (zod)
`sizeBytes`	`Int`	no	nou	Màxim 5 MB = 5242880
`width / height`	`Int?`	sí	nou	Píxels

Camp	Tipus (Prisma)	Null	Repo	Notes
`position`	`Int`	no	sí	Ordre a la galeria, 0-14. `@@unique([propertyId, position])` (nou)
`isCover`	`Boolean`	no	nou	Default false
`createdAt`	`DateTime`	no	nou	

Màxim 15 fotos per anunci: ho controla l'API. Per a la portada, un índex únic parcial `CREATE UNIQUE INDEX ... ON "PropertyImage"("propertyId") WHERE "isCover"` afegit a mà a la migració garanteix *com a màxim* una portada; que n'hi hagi *almenys* una (si hi ha fotos) ho ha de garantir l'API en esborrar o reordenar.

`Conversation` US-21, 22

Un fil per anunci i persona interessada. A S2 (US-21) n'hi ha prou amb crear la conversa i el primer missatge; a S3* el mateix model dona la safata "Els meus xats" (US-22).

Camp	Tipus (Prisma)	Null	Repo	Notes
`id`	`String`	no	nou	PK, cuid
`propertyId`	`String`	no	nou	FK → Property.id
`interestedUserId`	`String`	no	nou	FK → User.id. Qui escriu
`advertiserId`	`String`	no	nou	FK → User.id. Es copia de Property.ownerId en crear la conversa
`lastMessageAt`	`DateTime?`	sí	nou	Per ordenar la safata de xats
`createdAt` / `updatedAt`	`DateTime`	no	nou	

`@@unique([propertyId, interestedUserId])`. No es pot escriure al propi anunci (US-21): ho comprova la mutació i, opcionalment, un `CHECK ("interestedUserId" <> "advertiserId")` afegit a mà a la migració.

`Message` US-21, 22, 23

Camp	Tipus (Prisma)	Null	Repo	Notes
`id`	`String`	no	nou	PK, cuid
`conversationId`	`String`	no	nou	FK → Conversation.id
`senderId`	`String`	no	nou	FK → User.id
`body`	`String`	no	nou	No pot estar buit (US-21): `z.string().trim().min(1)`
`readAt`	`DateTime?`	sí	nou	Missatges no llegits (US-22) i avís per email (US-23)
`createdAt`	`DateTime`	no	nou	

`Favourite` US-14

Taula intermèdia usuari-anunci. Ja existeix al repo tal com cal.

Camp	Tipus (Prisma)	Null	Repo	Notes
`userId`	`String`	no	sí	FK → User.id
`propertyId`	`String`	no	sí	FK → Property.id
`createdAt`	`DateTime`	no	sí	

PK composta `@@id([userId, propertyId])`: un anunci no es pot desar dues vegades.

`Report` rol usuari/admin, T-05

Un usuari denuncia un anunci; un admin la revisa i decideix.

Camp	Tipus (Prisma)	Null	Repo	Notes
`id`	`String`	no	nou	PK, cuid
`propertyId`	`String`	no	nou	FK → Property.id

Camp	Tipus (Prisma)	Null	Repo	Notes
`reporterId`	`String?`	sí	nou	FK → User.id. Null si l'usuari esborra el compte
`reason`	`ReportReason`	no	nou	
`details`	`String?`	sí	nou	Text lliure de l'usuari
`status`	`ReportStatus`	no	nou	Default PENDING
`resolvedById`	`String?`	sí	nou	FK → User.id (admin)
`resolvedAt`	`DateTime?`	sí	nou	
`resolutionNote`	`String?`	sí	nou	Nota interna de l'admin
`createdAt / updatedAt`	`DateTime`	no	nou	

`@@unique([propertyId, reporterId])`: un usuari només pot denunciar una vegada el mateix anunci. A PostgreSQL els NULL no col·lideixen, així que les denúncies d'usuaris esborrats es conserven.

`UserPreference` US-15

Com a màxim una fila per usuari (1:1).

Camp	Tipus (Prisma)	Null	Repo	Notes
`userId`	`String`	no	nou	PK i FK → User.id
`listingType`	`ListingType?`	sí	nou	
`propertyTypes`	`PropertyType[]`	no	nou	Llista nativa de PostgreSQL. Default buida
`minPriceCents / maxPriceCents`	`Int?`	sí	nou	Cèntims
`minBedrooms`	`Int?`	sí	nou	
`minAreaM2`	`Int?`	sí	nou	
`city / province`	`String?`	sí	nou	
`createdAt / updatedAt`	`DateTime`	no	nou	

`ContactMessage` US-25

Missatges a l'equip de Dovivo; els gestiona un admin.

Camp	Tipus (Prisma)	Null	Repo	Notes
`id`	`String`	no	nou	PK, cuid
`userId`	`String?`	sí	nou	FK → User.id. Null si escriu un visitant
`name`	`String`	no	nou	
`email`	`String`	no	nou	Per respondre
`subject`	`String`	no	nou	Màx. 150 (zod)
`message`	`String`	no	nou	
`status`	`ContactStatus`	no	nou	Default OPEN
`createdAt / updatedAt`	`DateTime`	no	nou	

`PasswordResetToken` US-08 (Post)

Només aplica als usuaris amb contrasenya (Credentials). Es podria reutilitzar `VerificationToken`, però un model propi deixa clar què és cada token.

Camp	Tipus (Prisma)	Null	Repo	Notes
`id`	`String`	no	nou	PK, cuid
`userId`	`String`	no	nou	FK → User.id

Camp	Tipus (Prisma)	Null	Repo	Notes
tokenHash	String @unique	no	nou	Hash SHA-256 del token enviat per email (mai el token en clar)
expiresAt	DateTime	no	nou	Caduca als 60 min
createdAt	DateTime	no	nou	

SavedSearch US-16 (Post)

Camp	Tipus (Prisma)	Null	Repo	Notes
id	String	no	nou	PK, cuid
userId	String	no	nou	FK → User.id
name	String	no	nou	Nom que li dona l'usuari, màx. 100 (zod)
filters	Json	no	nou	Els mateixos paràmetres que la cerca. Es valida amb propertyFiltersSchema en desar i en llegir
alertsEnabled	Boolean	no	nou	Default true
lastAlertedAt	DateTime?	sí	nou	Per avisar només dels anuncis nous des de l'última alerta
createdAt / updatedAt	DateTime	no	nou	

5. Què passa en esborrar (onDelete)

Esborrar el compte (US-07, dret de supressió) és un esborrat real de l'usuari i de les seves dades. Esborrar un anunci des de l'app (US-19) és lògic (deletedAt); l'esborrat real d'anuncis només passa en cascada des de User. A Prisma es defineix amb onDelete a cada @relation.

Clau forana	onDelete	Motiu
Property.ownerId	Cascade	En esborrar el compte s'esborren els seus anuncis (ja al repo)
PropertyImage.propertyId	Cascade	L'API també ha d'esborrar els fitxers de l'emmagatzematge (ja al repo)
Favourite.userId / propertyId	Cascade	Ja al repo
Account.userId, Session.userId	Cascade	Ja al repo (Auth.js)
UserPreference.userId, SavedSearch.userId, PasswordResetToken.userId	Cascade	
Conversation.propertyId / interestedUserId / advertiserId	Cascade	Si un participant esborra el compte, el xat desapareix per als dos
Message.conversationId / senderId	Cascade	
Report.propertyId	Cascade	
Report.reporterId, Report.resolvedById	SetNull	Es conserva l'historial de moderació
ContactMessage.userId	SetNull	Es conserva la incidència

6. Índexs

A més de les PK i els @unique, cal un @@index a cada clau forana: PostgreSQL no en crea automàticament.

Model	Índex (Prisma)	Per a què
Property	@@index([status, listingType, propertyType])	Llistat públic i filtres (US-09, US-10). Substitueix l'actual [listingType, status, city]
Property	@@index([status, priceCents])	Filtre i ordre per preu (US-10, US-11)
Property	@@index([city]), [province], [postalCode], [neighborhood]	Cerca per ubicació (US-10)
Property	@@index([latitude, longitude])	Cerca per radi i mapa (US-13)

Model	Índex (Prisma)	Per a què
`Property`	`@@index([publishedAt])`, `[ownerId]`	Ordre 'més recents' · Els meus anuncis (ownerId ja existeix)
`Conversation`	`@@index([advertiserId, lastMessageAt])`, `[interestedUserId, lastMessageAt]`	Safata de xats (US-22)
`Message`	`@@index([conversationId, createdAt])`	Llegir un xat en ordre
`Report`	`@@index([status, createdAt])`	Cua de moderació de l'admin
`User`	`@@index([role])`	Llistats d'admins / agències

7. Procediments de l'API (tRPC)

L'API és tRPC, servida a `/api/trpc`. Cada domini és un router a `src/server/api/routers/`, registrat a `src/server/api/root.ts`. El frontend la crida amb els hooks tipats (`api.property.list.useQuery(...)`) i els Server Components amb el caller de servidor; no hi ha `fetch` manual a la nostra API.

- **Accés.** Públic = `publicProcedure`. Usuari = `protectedProcedure` (sessió d'Auth.js per cookie) i, si toca dades d'un altre, comprovació de propietat. Admin = `adminProcedure` (**nou** a `src/server/api/trpc.ts`): `protectedProcedure` + `role === "ADMIN"`.
- **Tipus.** *query* = lectura (GET), *mutation* = escriptura (POST).
- **Errors.** `TRPCError` amb codis estàndard: `BAD_REQUEST` (zod, amb errors per camp), `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`, `CONFLICT`, `TOO_MANY_REQUESTS`.
- **Inici i tancament de sessió** no són tRPC: els gestiona Auth.js (`/api/auth/*`) amb `signIn("credentials")` i `signOut()`.
- Els procediments marcats amb **(repo)** ja existeixen a `routers/property.ts`.

S1 (20 oct - 10 nov): comptes, explorar i fitxa

Procediment	Tipus	Accés	US	Què fa
`auth.signUp`	mutation	Públic	US-01	Registre amb nom, email i contrasenya. Desa `passwordHash`; després el client fa `signIn("credentials")` i la sessió queda iniciada
Auth.js `signIn("credentials")`	-	Públic	US-02	Inici de sessió. Error genèric si les credencials no són correctes; rebutja usuaris bloquejats
Auth.js `signOut()`	-	Usuari	US-03	Tanca la sessió (esborra la cookie)
`user.me`	query	Usuari	US-04	Dades del propi perfil
`user.updateMe`	mutation	Usuari	US-05	Editar nom, email i telèfon
`property.list` (repo)	query	Públic	US-09	Llistat d'anuncis publicats, amb filtre venda/lloguer. Paginació per cursor: `limit` per defecte 20, màx. 50; respon `{ items, nextCursor }`
`property.byId` (repo)	query	Públic	US-12	Fitxa de l'anunci amb fotos i anunciant. NOT_FOUND si no existeix o és DRAFT d'un altre

S2 (10 - 24 nov): publicar, cercar i contactar

Procediment	Tipus	Accés	US	Què fa
`property.list`	query	Públic	US-10	S'hi afegeixen filtres: preu, habitacions, m², diversos tipus, ciutat, barri, extres; i el total de resultats
`property.create` (repo)	mutation	Usuari	US-17	Crea l'anunci en DRAFT
`property.publish` (repo)	mutation	Usuari	US-17	Passa a PUBLISHED. Només el propietari
`property.photos.createUpload`	mutation	Usuari	US-18	Valida tipus (JPG/PNG), mida (≤ 5 MB) i límit de 15, i retorna una URL de pujada signada a l'emmagatzematge (T-10). Només el propietari
`property.photos.confirm`	mutation	Usuari	US-18	Registra la foto pujada (PropertyImage)
`property.photos.update`	mutation	Usuari	US-18	Canviar l'ordre o marcar com a portada
`property.photos.delete`	mutation	Usuari	US-18	Eliminar una foto (i el fitxer)
`conversation.start`	mutation	Usuari	US-21	Envia un missatge a l'anunciant. Crea la conversa si no existeix i envia l'email

S3 (24 nov - 15 des): preferits, gestió d'anuncis i moderació

Procediment	Tipus	Accés	US	Què fa
`property.mine` (repo)	query	Usuari	US-19	Els meus anuncis amb el seu estat
`property.update`	mutation	Usuari	US-19	Editar un anunci propi
`property.delete`	mutation	Usuari	US-19	Esborrat lògic (`deletedAt`) d'un anunci propi
`favourite.list`	query	Usuari	US-14	Llista de preferits, indicant els que ja no estan disponibles
`favourite.add` / `favourite.remove`	mutation	Usuari	US-14	Afegir o treure dels preferits (idempotent)
`report.create`	mutation	Usuari	Moderació	Denunciar un anunci
`admin.reports.list`	query	Admin	Moderació	Cua de denúncies (filtre per estat)
`admin.reports.resolve`	mutation	Admin	Moderació	Resoldre o descartar; pot bloquejar l'anunci
`admin.users.list`	query	Admin	Moderació	Llistat d'usuaris
`admin.users.setBlocked`	mutation	Admin	Moderació	Bloquejar o desbloquejar un usuari

S3* (si hi ha capacitat)

Procediment	Tipus	Accés	US	Què fa
`user.deleteAccount`	mutation	Usuari	US-07	Esborrar el compte i totes les dades (i els fitxers de fotos)
`property.list`	query	Públic	US-11	S'hi afegeix l'ordre (preu, m², data)
`property.nearby`	query	Públic	US-13	Anuncis dins d'un radi (màx. 50 km) per al mapa
`property.setStatus`	mutation	Usuari	US-20	Marcar com a reservat o tancat (venut/llogat)
`preference.get` / `preference.set`	query / mutation	Usuari	US-15	Consultar i desar les preferències
`conversation.list`	query	Usuari	US-22	Els meus xats, ordenats per l'últim missatge
`conversation.messages`	query	Usuari	US-22	Missatges d'un xat. Només els participants
`conversation.reply`	mutation	Usuari	US-22	Respondre en un xat
`conversation.markRead`	mutation	Usuari	US-22	Marcar els missatges com a llegits
`contact.send`	mutation	Públic	US-25	Enviar un missatge a l'equip de Dovivo
`admin.contact.list` / `setStatus`	query / mutation	Admin	US-25	Bústia de contacte i canvi d'estat

Post MVP

Procediment	Tipus	Accés	US	Què fa
`auth.forgotPassword`	mutation	Públic	US-08	Envia l'email per recuperar la contrasenya. Respon igual existeixi o no l'email
`auth.resetPassword`	mutation	Públic	US-08	Desa la nova contrasenya amb el token rebut
`auth.verifyEmail / resendVerification`	mutation	Públic / Usuari	US-06	Verificació de l'email amb `VerificationToken`
`savedSearch.list / create / update / delete`	query / mutation	Usuari	US-16	Cerques desades i alertes
`assistant.search`	mutation	Públic	US-24	Assistent de cerca: converteix text en filtres i retorna anuncis (secció 9)

L'avís per email de missatges nous (US-23) no té procediment propi: s'envia des de `conversation.start` i `conversation.reply`. Pujar fotos dins de tRPC no és pràctic (no gestiona multipart): per això el flux recomanat és URL signada + confirmació. Si T-10 tria un emmagatzematge sense URL signades, s'usarà un Route Handler de Next.js.

Contracte API (T-06)

Amb tRPC el frontend importa el tipus `AppRouter`: paràmetres, respostes i errors de validació són tipats i el compilador avisa de qualsevol desajust, així que **no cal un openapi.yaml** mentre l'únic client sigui el nostre frontend. El contracte són els esquemes zod de `src/lib/validators`. Queda per acordar entre frontend i backend:

- Paginació: per cursor (`{ items, nextCursor }`, ja implementat) o per pàgines numerades per a la US-11.
- Idioma i format dels missatges d'error que es mostren a l'usuari (els de zod per camp i els de negoci).
- Imports en cèntims i dates com a Date (superjson); el frontend formata en català/castellà.
- Si mai hi ha un client extern (app mòbil, partners), es pot generar OpenAPI des dels routers amb `trpc-to-openapi`.

8. Spike T-07: motor de cerca

Requisits del backlog

- US-09: el llistat es mostra en menys de 3 segons amb 1.000 anuncis.
- US-10: filtres combinables (operació, preu, habitacions, m², tipus, ciutat o barri), nombre de resultats, sense recarregar la pàgina i amb els filtres a la URL.
- US-11: ordenar i paginar. US-13: veure els anuncis en un mapa (cerca per zona).

Què hi ha ara a l'API

- `property.list` rep els filtres validats amb `propertyFiltersSchema`: `listingType`, `propertyType`, `city` (sense distingir majúscules), `minPriceCents`/`maxPriceCents` i `minBedrooms`. Els filtres es poden posar a la URL de la pàgina (/buy, /rent) i passar-los a la query.
- Paginació per cursor, ordenada per `publishedAt` descendent. **Cal afegir** `id` com a desempat a l'`orderBy`: sense, dos anuncis amb la mateixa data poden repetir-se o saltar-se entre pàgines.
- **Falta:** el total de resultats per al comptador de la US-10 (`ctx.db.property.count` amb el mateix `where`), diversos tipus alhora, m², barri, extres, ordre (US-11), text lliure i `property.nearby`.
- Text lliure (`q`): `contains` amb `mode: "insensitive"` sobre títol i descripció.
- `property.nearby`: prefiltre per quadrat amb Prisma (aprofita l'índex de latitude/longitude) i distància de Haversine; o tot en una consulta amb `$queryRaw` tipada.

Opcions avaluades

Opció	Avantatges	Inconvenients	Per al MVP
A. PostgreSQL + Prisma amb índexs	Cap servei nou. Filtres exactes i ràpids amb els índexs de la secció 6. Ja funciona	El text lliure amb contains no ordena per rellevància ni tolera errors	Sí
B. Full-text de PostgreSQL (tsvector)	Millor cerca de text sense servei nou	Prisma no ho suporta del tot a PostgreSQL: cal `$queryRaw` i una columna/índex GIN a la migració	Pla B si el text es queda curt
C. Meilisearch / Typesense	Tolerància a errors, rellevància, facetes i geo	Un servei més a desplegar i a mantenir sincronitzat des de les mutacions de tRPC	Post MVP
D. Elasticsearch / OpenSearch	Molt potent i escalable	Excessiu per al volum del MVP; cost d'operació alt	No

Opció personal escollida, a debatre amb l'equip: A, PostgreSQL amb índexs.

Amb uns milers d'anuncis, una base de dades amb els índexs adequats respon en pocs mil·lisegons, molt per sota dels 3 segons de la US-09. Ens estalviem un servei més a S0, que ja va carregat per a DevOps (T-01 a T-03).

- **Majúscules i accents:** `mode: "insensitive"` només ignora majúscules. Perquè «lleida» trobi «Lleida» i «Sant Marti» trobi «Sant Martí», la proposta és desar columnes normalitzades (`citySearch`, `neighborhoodSearch`: minúscules i sense accents, calculades a l'API en desar) i filtrar per elles, sense SQL cru. Alternativa: l'extensió `unaccent` activada a la migració i consultes amb `$queryRaw`.
- **Geocodificació:** convertir l'adreça en latitud i longitud és del spike T-08 (mapes); la cerca només fa servir les coordenades.
- **Quan canviar a C:** si passem de ~50.000 anuncis, si el temps de resposta supera els 500 ms o si negoci vol tolerància a errors d'escriptura.

Com ho validem

- Carregar 10.000 anuncis de prova amb un seed de Prisma (`prisma/seed.ts`, T-15) i mesurar el temps de `property.list` amb combinacions habituals de filtres.
- Objectiu: p95 per sota de 300 ms a l'API. Revisar amb `EXPLAIN ANALYZE` que les consultes fan servir els índexs.

9. Spike T-09: viabilitat del xatbot

Què hauria de fer

La US-24 (Could, 8 SP, valor 3) demana un xatbot per resoldre dubtes generals sense esperar una persona: com publicar, com contactar, què vol dir cada estat d'un anunci... Quan no sàpiga la resposta, ha de derivar al formulari de contacte (US-25).

Opcions avaluades

Opció	Avantatges	Inconvenients	Esforç
A. FAQ estàtica amb cercador (sense IA)	Cost zero, sense riscos	No entén preguntes formulades d'una altra manera	2-3 SP
B. Widget SaaS (Tidio, Intercom...)	Llest en poc temps	Quota mensual, dades a tercers, no coneix els anuncis	1-2 SP
C. Model de llenguatge (API de Claude) amb la FAQ com a context	Respostes naturals, en català i castellà	Cost per ús; cal evitar que s'inventi coses	5 SP
D. Com C, però pot fer cerques d'anuncis	Respon «pisos a Girona per menys de 800 €» amb anuncis reals	Més complex de fer i de provar	8+ SP
E. Model obert en local (Ollama) per a la demo	Gratuït, sense compte ni targeta, les dades no surten de l'ordinador	Cal un ordinador amb 16 GB de RAM; més lent i amb pitjor català. No serveix per a producció	2-3 SP

Opció E, per a la demo: Ollama executa models oberts (p. ex. Llama o Qwen) al mateix ordinador de la demo, sense cap cost. El servidor de Next.js hi parla per HTTP igual que parlaria amb l'API de Claude, així que n'hi ha prou amb fer configurable el proveïdor amb una variable d'entorn (p. ex. `ASSISTANT_PROVIDER=ollama|anthropic`, afegida a `.env.example` i a `src/env.js`): per a la demo, Ollama; per a producció, Claude (opció C o D), sense reescriure codi.

Cost estimat

Suposant uns 4.000 tokens d'entrada (FAQ + conversa) i 300 de sortida per missatge, sense prompt caching. Preus de l'API d'Anthropic en dòlars (setembre de 2026).

Model	Entrada / 1M tokens	Sortida / 1M tokens	Cost aprox. per 1.000 missatges
Claude Haiku 4.5	1 $	5 $	~5,5 $
Claude Sonnet 5.5	2 $	10 $	~11 $
Claude Opus 5.5	4 $	20 $	~22 $ + raonament intern
Model obert en local (Ollama)	0 $	0 $	0 $ (només l'ordinador)

Amb prompt caching, la FAQ només es cobra sencera la primera vegada, i el cost d'entrada baixa molt. Per a l'ús esperat d'un MVP (centenars de missatges al mes), el cost és de pocs dòlars al mes en qualsevol cas.

Riscos

- **Respostes inventades** (anuncis, preus o normes que no existeixen): només ha de respondre amb la FAQ i amb dades reals de l'API, i derivar a la US-25 quan no ho sàpiga.
- **Privacitat (RGPD):** no s'hi han d'enviar dades personals d'altres usuaris; cal informar l'usuari que parla amb una IA.
- **Abús i cost:** límit de missatges per usuari i dia, mida màxima de resposta i alerta de despesa al compte de l'API. tRPC no porta limitador: cal un middleware (`rateLimitedProcedure`) amb un comptador a memòria (una instància) o a Redis/Upstash (diverses).

Opció personal escollida, a debatre amb l'equip: D, assistent de cerca

L'assistent rep una petició en text lliure, la converteix en els filtres de `property.list` i retorna els pisos que més hi coincideixen. El model no accedeix a la base de dades ni escriu SQL: només proposa filtres, i l'API els valida amb les mateixes regles que la cerca normal i fa la consulta. Només llegeix anuncis publicats.

Pas	Què passa	Exemple
1	L'usuari escriu la petició	«Pis de lloguer a Girona, 2 habitacions, menys de 900 € i amb ascensor»
2	El servidor l'envia al model amb una sola eina, `search_properties`, l'esquema de la qual es genera a partir de `propertyFiltersSchema` (zod → JSON Schema, p. ex. amb `zod-to-json-schema`)	`listingType, propertyTypes[], city, neighborhood, minPriceCents, maxPriceCents, minAreaM2, minBedrooms, minBathrooms, hasElevator, hasParking, isFurnished, sort`
3	El model respon amb els filtres	`{ listingType: "RENT", propertyTypes: ["APARTMENT"], city: "Girona", minBedrooms: 2, maxPriceCents: 90000, hasElevator: true }`
4	L'API valida els filtres amb `propertyFiltersSchema.safeParse` i fa la consulta amb el mateix codi que `property.list`, amb un màxim de 10 anuncis	Mateix codi que la cerca normal
5	Si no hi ha resultats, l'API relaxa els filtres en un ordre fix i ho indica	Preu +10%, després habitacions -1, després treu els extres
6	Es retornen els filtres aplicats, què s'ha relaxat i els anuncis	El frontend mostra les mateixes targetes que al llistat

- **Procediment:** `assistant.search` (mutation, públic) amb `{ message }`. Respon `{ filters, relaxed, message, properties }`. Límit de 10 consultes per minut.
- **«Els que més coincideixen»:** primer els que compleixen tots els filtres; si no n'hi ha, els de la cerca relaxada, dient a l'usuari què s'ha relaxat. L'ordre és el de la cerca normal (sort), o el més recent si no se n'indica cap.

- **Criteris que no són filtres** (p. ex. «zona bona», «a prop de la línia S1»): el model no els pot aplicar. L'assistent ho diu en la resposta, en lloc d'ignorar-ho en silenci.
- **Una sola crida al model per cerca:** el missatge de resposta el munta el servidor amb una plantilla a partir dels filtres. Amb Claude Haiku 4.5 són uns 1.500 tokens d'entrada i 200 de sortida, uns 2,5 $ per cada 1.000 cerques.
- **Clau de l'API:** `ANTHROPIC_API_KEY` només al servidor (`src/env.js`, bloc `server`), mai amb prefix `NEXT_PUBLIC_`.
- **Demo gratuïta:** el mateix codi amb Ollama (opció E), fent servir un model obert que admeti eines (p. ex. Llama 3.1 o Qwen 2.5). Per a producció es canvia el proveïdor a Claude per configuració.
- **Quan:** Post MVP, coherent amb el backlog (Could, valor 3). Es pot ensenyar en una demo de sprint amb Ollama sense cost. Cal tornar a estimar la US-24 amb aquest abast (orientativament 5-8 SP).
- **No cal cap model.** Si es fa una prova amb usuaris, convé desar les peticions i els filtres generats per avaluar-los (un model `AssistantQuery`).

10. Decisions pendents

#	Decisió	Proposta d'aquest document
1	Afegir `COMMERCIAL` i `GARAGE` a `PropertyType`	Sí, el backlog els preveu
2	Afegir `BLOCKED` a `ListingStatus`	Sí, cal per a la moderació (T-05)
3	Login: Credentials (email + contrasenya) i/o OAuth	Credentials obligatori per a US-01/02; OAuth (Discord, Google) opcional
4	Longitud màxima de la descripció: 5.000 (zod actual) o 10.000 (backlog)	Decidir i alinear el validador
5	Paginació de la US-11: cursor o pàgines numerades	Cursor (ja implementat) amb «carregar més»; pàgines només si UX ho demana
6	Cerca sense accents: columnes normalitzades o unaccent	Columnes normalitzades (sense SQL cru)
7	Emmagatzematge d'imatges (T-10)	Un servei amb URL signades (S3, R2...) per al flux de la secció 7